

i will give you leetcode questions you have to do

1. explain the questions in simple points
2. explain the brute force approach in points with time complexity and space
3. give the code in c++ of that approach
4. explain better approach in points with time complexity and space
5. give the code in c++ of that approach
6. explain optimal approach in points time complexity and space
7. give the code in c++ of that approach

Understood.

For every LeetCode problem you send, I will follow exactly this format:

1. Problem Explanation (Simple English)

- What is given?
- What is required?
- Small example
- Why brute force may fail

2. Brute Force Approach

Idea

- Step-by-step explanation in points

Time Complexity

- $O(\dots)$

Space Complexity

- $O(\dots)$

C++ Code

```
</> C++
```



```
// Brute Force
```

3. Better Approach

Idea

- Step-by-step explanation in points

Why it is better?

Time Complexity

- $O(\dots)$

Space Complexity

- $O(\dots)$

C++ Code

```
</> C++
```



```
// Better Approach
```

4. Optimal Approach

Core Observation / Trick

- Key insight
- What most people miss

Idea

- Step-by-step explanation

Edge Cases

- Mention important cases

Time Complexity

- $O(\dots)$

Space Complexity

- $O(\dots)$

C++ Code

```
</> C++
```



```
// Optimal Approach
```

5. Dry Run

- Take a small example
- Show variables changing step by step
- Explain how answer is obtained

6. If Problem is Hard (1800+/LeetCode Hard)

Additionally:

- Hidden observation
- Alternative ways to think
- Common mistakes
- Interview discussion points

Send the first LeetCode problem.

     ...  Sources

Longest Substring Without Repeating Characters

Longest Substring Without Repeating Characters (LeetCode 3)

1. Problem Explanation (Simple English)

What is given?

A string `s`.

What is required?

Find the length of the **longest substring** that contains **no repeating characters**.

Example

```
s = "abcabcbb"
```



Possible substrings without repeating characters:

```
"abc" -> length 3
```

```
"bca" -> length 3
```

```
"cab" -> length 3
```



Answer:



Why brute force may fail?

For every starting position, we can try all ending positions and check whether characters repeat.

This becomes very slow for large strings because there are $O(n^2)$ substrings.

2. Brute Force Approach

Idea

- Generate every substring.
- For each substring:
 - Check if all characters are unique using a set.
- If unique:
 - Update answer.

Time Complexity

There are $O(n^2)$ substrings.

Checking uniqueness takes $O(n)$.

Time: $O(n^3)$

Space: $O(n)$



C++ Code

</> C++



```

class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        int n = s.size();
        int ans = 0;

        for(int i = 0; i < n; i++) {
            for(int j = i; j < n; j++) {
                unordered_set<char> st;
                bool ok = true;

                for(int k = i; k <= j; k++) {
                    if(st.count(s[k])) {
                        ok = false;
                        break;
                    }
                    st.insert(s[k]);
                }

                if(ok) {
                    ans = max(ans, j - i + 1);
                }
            }
        }

        return ans;
    }
};

```

3. Better Approach

Idea

Instead of checking every substring again and again:

- Fix starting index **i**.
- Extend substring to the right.
- Maintain a set of characters.
- Stop when a duplicate appears.

Steps

For every **i**:

- Create empty set.
- Move **j** from **i** to **n-1**.
- If character already exists:
 - Break.

- Otherwise:
 - Insert character.
 - Update answer.

Why Better?

We avoid scanning the same substring repeatedly.

Time Complexity

Time: $O(n^2)$

Space: $O(n)$



C++ Code

 C++



```
class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        int n = s.size();
        int ans = 0;

        for(int i = 0; i < n; i++) {
            unordered_set<char> st;

            for(int j = i; j < n; j++) {
                if(st.count(s[j])) {
                    break;
                }

                st.insert(s[j]);
                ans = max(ans, j - i + 1);
            }
        }

        return ans;
    }
};
```

4. Optimal Approach (Sliding Window)

Core Observation / Trick

If a character repeats:

abcdea

^



Then every substring containing both **a**s is invalid.

Instead of restarting, we can move the left pointer forward and maintain a valid window.

This is the classic **Sliding Window** technique.

Idea

Maintain:

[left ... right]



such that all characters are unique.

For every character:

- Add current character.
 - If duplicate occurs:
 - Move left pointer until duplicate is removed.
 - Update answer using current window size.
-

Steps

1. Start with `left = 0`.
 2. Move `right` from `0` to `n-1`.
 3. Store character frequencies.
 4. While frequency of current character > 1 :
 - Remove left character.
 - Increment left.
 5. Window becomes valid again.
 6. Update answer.
-

Edge Cases

Empty String

```
s = ""  
answer = 0
```



Single Character

```
s = "a"  
answer = 1
```



All Same Characters

```
s = "aaaa"  
answer = 1
```



All Unique

```
s = "abcdef"  
answer = 6
```



Time Complexity

Each character enters window once and leaves once.

```
Time:  $O(n)$   
Space:  $O(256) \approx O(1)$ 
```



C++ Code

```
</> C++
```




```
class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        vector<int> freq(256, 0);

        int left = 0;
        int ans = 0;

        for(int right = 0; right < s.size(); right++) {
            freq[s[right]]++;

            while(freq[s[right]] > 1) {
                freq[s[left]]--;
                left++;
            }

            ans = max(ans, right - left + 1);
        }

        return ans;
    }
};
```

5. Dry Run

Example:

```
s = "abcabcbb"
```



Initial

```
left = 0
ans = 0
```



right = 0 ('a')

```
window = "a"
ans = 1
```



right = 1 ('b')

```
window = "ab"
ans = 2
```



right = 2 ('c')

```
window = "abc"  
ans = 3
```



right = 3 ('a')

Duplicate found.

Move left:

```
remove 'a'  
left = 1  
  
window = "bca"  
ans = 3
```



right = 4 ('b')

Duplicate.

```
remove 'b'  
left = 2  
  
window = "cab"  
ans = 3
```



right = 5 ('c')

Duplicate.

```
remove 'c'  
left = 3  
  
window = "abc"  
ans = 3
```



right = 6 ('b')

Duplicate.

```
remove 'a'
left = 4

remove 'b'
left = 5

window = "cb"
ans = 3
```



right = 7 ('b')

Duplicate.

```
remove 'c'
left = 6

remove 'b'
left = 7

window = "b"
```



Final Answer:

3



Common Interview Follow-up

Can we directly jump `left` instead of moving it one-by-one?

Yes.

Store the **last index** of each character using a hashmap.

This also gives $O(n)$ time and is another popular optimal solution asked in interviews.



Sources

Longest Repeating Character Replacement

Longest Repeating Character Replacement (LeetCode 424)

1. Problem Explanation (Simple English)

What is given?

- A string `s` containing only uppercase English letters.
- An integer `k`.

What is required?

You can change at most `k` characters in any substring.

Find the length of the **longest substring** that can be converted into a string containing only one repeating character.

Example

```
s = "ABAB"  
k = 2
```



Change both 'A' to 'B' (or both 'B' to 'A'):

```
ABAB → BBBB
```



Answer:

```
4
```



Example

```
s = "AABABBA"  
k = 1
```



Longest valid substring:

```
AABA
```



Change one 'B' → 'A'

```
AAAA
```



Answer:



Why brute force may fail?

There are $O(n^2)$ substrings.

Checking every substring separately becomes too slow.

2. Brute Force Approach

Idea

Generate every substring.

For each substring:

- Count frequency of all characters.
- Find maximum frequency character.
- Characters to replace:

```
substring_length - max_frequency
```



If:

```
substring_length - max_frequency <= k
```



then substring is valid.

Update answer.

Time Complexity

- $O(n^2)$ substrings
- $O(n)$ checking each substring

```
Time:  $O(n^3)$   
Space:  $O(26)$ 
```



</> C++



```
class Solution {
public:
    int characterReplacement(string s, int k) {
        int n = s.size();
        int ans = 0;

        for(int i = 0; i < n; i++) {
            for(int j = i; j < n; j++) {

                vector<int> freq(26, 0);

                for(int p = i; p <= j; p++) {
                    freq[s[p] - 'A']++;
                }

                int mx = 0;

                for(int x : freq) {
                    mx = max(mx, x);
                }

                int len = j - i + 1;

                if(len - mx <= k) {
                    ans = max(ans, len);
                }
            }
        }

        return ans;
    }
};
```

3. Better Approach

Idea

For every starting index:

- Extend substring to the right.
- Maintain frequency array incrementally.
- Keep track of maximum frequency.

Instead of recalculating frequencies from scratch.

Steps

For every `i`:

- Create freq array.
- Move `j` from `i` to `n-1`.
- Update frequency.
- Update max frequency.
- Check validity:

```
window_length - max_frequency <= k
```



- Update answer.

Time Complexity

Time: $O(n^2)$

Space: $O(26)$



C++ Code

 C++



```

class Solution {
public:
    int characterReplacement(string s, int k) {
        int n = s.size();
        int ans = 0;

        for(int i = 0; i < n; i++) {

            vector<int> freq(26, 0);
            int mxFreq = 0;

            for(int j = i; j < n; j++) {

                freq[s[j] - 'A']++;

                mxFreq = max(mxFreq, freq[s[j] - 'A']);

                int len = j - i + 1;

                if(len - mxFreq <= k) {
                    ans = max(ans, len);
                }
            }
        }

        return ans;
    }
};

```

4. Optimal Approach (Sliding Window)

Core Observation / Trick

Suppose window is:

AABAB



Frequency:

A = 3

B = 2



Most frequent character count:

maxFreq = 3



To make entire window same:


```
window_size - maxFreq
```



characters must be replaced.

Valid Window Condition

```
window_size - maxFreq <= k
```



If this condition fails:

```
window_size - maxFreq > k
```



we must shrink the window.

Idea

Maintain:

```
[left ... right]
```



Keep:

- frequency array
- maximum frequency in current window

Expand right.

If invalid:

```
window_size - maxFreq > k
```



move left.

Keep updating answer.

Steps

1. Add current character.
2. Update `maxFreq`.
3. Check:

```
(right-left+1) - maxFreq > k
```



4. If true:
 - shrink window.
5. Update answer.

Edge Cases

$k = 0$

Need already repeating characters only.

```
ABAB  
answer = 1
```



Entire String Convertible

```
ABAB  
k = 2  
  
answer = 4
```



Single Character

```
A  
  
answer = 1
```



Time Complexity

Each pointer moves once.

```
Time:  $O(n)$   
Space:  $O(26) = O(1)$ 
```



C++ Code

⌞ C++



```
class Solution {
public:
    int characterReplacement(string s, int k) {
        vector<int> freq(26, 0);

        int left = 0;
        int maxFreq = 0;
        int ans = 0;

        for(int right = 0; right < s.size(); right++) {

            freq[s[right] - 'A']++;

            maxFreq = max(maxFreq, freq[s[right] - 'A']);

            while((right - left + 1) - maxFreq > k) {
                freq[s[left] - 'A']--;
                left++;
            }

            ans = max(ans, right - left + 1);
        }

        return ans;
    }
};
```

5. Dry Run

Example:

```
s = "AABABBA"
k = 1
```



right = 0

```
window = A
maxFreq = 1

ans = 1
```



right = 1

```
window = AA  
maxFreq = 2
```



```
2 - 2 = 0 <= 1
```

```
ans = 2
```

right = 2

```
window = AAB  
maxFreq = 2
```



```
3 - 2 = 1 <= 1
```

```
ans = 3
```

right = 3

```
window = AABA  
maxFreq = 3
```



```
4 - 3 = 1 <= 1
```

```
ans = 4
```

right = 4

```
window = AABAB  
maxFreq = 3
```



```
5 - 3 = 2 > 1
```

Invalid.

Shrink:

```
remove first A
```



```
window = ABAB
```

```
left++
```

Now valid.

Continue similarly.

Final answer:

4



Common Interview Trick

Many candidates think `maxFreq` must be decreased when shrinking the window.

It is **not necessary**.

Keeping an old (possibly larger) `maxFreq` still guarantees correctness and preserves $O(n)$ complexity. This is the key observation most people miss in this problem.



Sources

Permutation In String

Permutation in String (LeetCode 567)

1. Problem Explanation (Simple English)

What is given?

- String `s1`
- String `s2`

What is required?

Check whether `s2` contains any substring that is a **permutation (anagram)** of `s1`.

Return:

```
true -> if any permutation exists  
false -> otherwise
```



Example 1

```
s1 = "ab"  
s2 = "eidbaooo"
```



Substring:

```
"ba"
```



is a permutation of:

```
"ab"
```



Answer:

```
true
```



Example 2

```
s1 = "ab"  
s2 = "eidboao"
```



No substring of length 2 is a permutation of "ab".

Answer:

```
false
```



Key Observation

If a substring is a permutation of `s1`, then:

```
Length must be same as s1
Character frequencies must be same
```



Example:

```
ab
ba
```



Frequency:

```
a = 1
b = 1
```



Same frequencies $\Rightarrow$ permutation.

Why brute force may fail?

For every substring of length `|s1|`:

- Generate frequency
- Compare with `s1`

This becomes expensive for large strings.

2. Brute Force Approach

Idea

Generate every substring of length `s1.size()`.

For each substring:

- Sort substring
- Sort `s1`
- Compare

If equal:

```
return true
```



Else continue.

Time Complexity

Let:

```
n = s2.length()
m = s1.length()
```



For every window:

```
sorting = O(m log m)
```



Total:

```
Time: O((n-m+1) * m log m)
Space: O(m)
```



C++ Code

 C++



```
class Solution {
public:
    bool checkInclusion(string s1, string s2) {

        int m = s1.size();

        string target = s1;
        sort(target.begin(), target.end());

        for(int i = 0; i + m <= s2.size(); i++) {

            string cur = s2.substr(i, m);
            sort(cur.begin(), cur.end());

            if(cur == target)
                return true;
        }

        return false;
    }
};
```

3. Better Approach

Idea

Instead of sorting:

- For every substring of length `m`
- Count frequencies of 26 letters
- Compare with frequency of `s1`

Frequency comparison is faster than sorting.

Steps

1. Store frequency of `s1`.
 2. For every window of size `m`:
 - Build frequency array.
 - Compare with `s1` frequency.
 3. If equal:
 - return true
-

Time Complexity

There are $O(n)$ windows.

Each comparison:

26 characters



Window construction:

$O(m)$



Overall:

Time: $O((n-m+1) * m)$

Space: $O(26)$



C++ Code

`</>` C++



```

class Solution {
public:
    bool checkInclusion(string s1, string s2) {

        int m = s1.size();

        vector<int> need(26, 0);

        for(char c : s1)
            need[c - 'a']++;

        for(int i = 0; i + m <= s2.size(); i++) {

            vector<int> freq(26, 0);

            for(int j = i; j < i + m; j++) {
                freq[s2[j] - 'a']++;
            }

            if(freq == need)
                return true;
        }

        return false;
    }
};

```

4. Optimal Approach (Sliding Window)

Core Observation / Trick

We only care about windows of size:

```
m = s1.length()
```



Instead of rebuilding frequency for every window:

- Add one character entering the window.
- Remove one character leaving the window.

This is a classic **fixed-size sliding window**.

Idea

Maintain:

```
need = frequency of s1  
window = frequency of current window in s2
```



Window size always remains:

```
m
```



At each step:

- Add new character.
- If size exceeds m:
 - Remove left character.
- Compare frequencies.

If equal:

```
return true
```



Steps

1. Build frequency of `s1`.
2. Traverse `s2`.
3. Add current character.
4. If window size > m:
 - remove left character.
5. Compare frequency arrays.
6. If equal:
 - return true.

Edge Cases

s1 larger than s2

```
s1 = "abcd"  
s2 = "ab"
```



Impossible.

Return:

```
false
```



Exact Match

```
s1 = "abc"  
s2 = "abc"
```



Answer:

```
true
```



Repeated Characters

```
s1 = "aab"  
s2 = "eidbaao"
```



Must match exact frequencies.

Time Complexity

Each character enters and leaves window once.

Frequency comparison:

```
26
```



constant.

```
Time:  $O(n)$   
Space:  $O(26) = O(1)$ 
```



C++ Code

```
</> C++
```



```

class Solution {
public:
    bool checkInclusion(string s1, string s2) {

        int m = s1.size();
        int n = s2.size();

        if(m > n)
            return false;

        vector<int> need(26, 0);
        vector<int> window(26, 0);

        for(char c : s1)
            need[c - 'a']++;

        int left = 0;

        for(int right = 0; right < n; right++) {

            window[s2[right] - 'a']++;

            if(right - left + 1 > m) {
                window[s2[left] - 'a']--;
                left++;
            }

            if(window == need)
                return true;
        }

        return false;
    }
};

```

5. Dry Run

Input

```

s1 = "ab"
s2 = "eidbaooo"

```



Need Frequency

```

a = 1
b = 1

```



Window = "ei"

```
e=1  
i=1
```



Not equal.

Window = "id"

```
i=1  
d=1
```



Not equal.

Window = "db"

```
d=1  
b=1
```



Not equal.

Window = "ba"

```
b=1  
a=1
```



Matches `s1`.

Return:

```
true
```



Common Interview Trick

Many people try to generate all permutations of `s1`.

For example:

```
s1 = "abcdef"
```



Number of permutations:

```
6! = 720
```



For larger lengths:

```
10! = 36,28,800
```



This is impossible.

The correct observation is:

```
Two strings are permutations  
⇒ Their frequency arrays are identical.
```



This converts the problem into a simple **fixed-size sliding window + frequency counting** problem with **$O(n)$** time.



Sources

Minimum Size Subarray Sum

Minimum Size Subarray Sum (LeetCode 209)

1. Problem Explanation (Simple English)

What is given?

- An array `nums` of positive integers.
- An integer `target`.

What is required?

Find the **minimum length** of a contiguous subarray whose sum is **greater than or equal to target**.

If no such subarray exists, return:

0



Example

```
target = 7  
nums = [2,3,1,2,4,3]
```



Subarrays with sum ≥ 7 :

```
[2,3,1,2] -> length 4  
[3,1,2,4] -> length 4  
[4,3]     -> length 2
```



Minimum length:

2



Answer:

2



Why brute force may fail?

There are $O(n^2)$ subarrays.

Checking the sum of every subarray separately can lead to $O(n^3)$ time.

2. Brute Force Approach

Idea

- Generate all possible subarrays.
- Compute their sum.
- If sum $\geq$ target:
 - Update minimum length.

Time Complexity

Time: $O(n^3)$

Space: $O(1)$



C++ Code

</> C++



```
class Solution {
public:
    int minSubArrayLen(int target, vector<int>& nums) {
        int n = nums.size();
        int ans = INT_MAX;

        for(int i = 0; i < n; i++) {
            for(int j = i; j < n; j++) {

                int sum = 0;

                for(int k = i; k <= j; k++) {
                    sum += nums[k];
                }

                if(sum >= target) {
                    ans = min(ans, j - i + 1);
                }
            }
        }

        return ans == INT_MAX ? 0 : ans;
    }
};
```

3. Better Approach

Idea

Instead of recomputing the sum every time:

- Fix starting index **i**.
- Extend ending index **j**.
- Maintain running sum.

Steps

For every `i`:

- `sum = 0`
- Move `j` from `i` to `n-1`
- Add `nums[j]`
- If `sum ≥ target`:
 - Update answer

Time Complexity

Time: $O(n^2)$

Space: $O(1)$



C++ Code

 C++



```
class Solution {
public:
    int minSubArrayLen(int target, vector<int>& nums) {
        int n = nums.size();
        int ans = INT_MAX;

        for(int i = 0; i < n; i++) {

            int sum = 0;

            for(int j = i; j < n; j++) {

                sum += nums[j];

                if(sum >= target) {
                    ans = min(ans, j - i + 1);
                }
            }
        }

        return ans == INT_MAX ? 0 : ans;
    }
};
```

4. Optimal Approach (Sliding Window)

Core Observation / Trick

The array contains **only positive integers**.

This is the key observation.

Because all numbers are positive:

```
If we expand the window,  
sum can only increase.
```



```
If we shrink the window,  
sum can only decrease.
```

Therefore, sliding window works perfectly.

Idea

Maintain:

```
[left ... right]
```



and keep track of the current window sum.

Expand

Move `right` and add elements.

Shrink

Whenever:

```
sum >= target
```



the current window is valid.

Try to make it smaller by moving `left`.

Update answer while shrinking.

Steps

1. Start with:

```
left = 0  
sum = 0
```



2. Move `right` from `0` to `n-1`.
3. Add:

```
sum += nums[right]
```



4. While:

```
sum >= target
```



- Update answer
- Remove `nums[left]`
- Move `left`

5. Continue.

Edge Cases

No Valid Subarray

```
target = 100  
nums = [1,2,3]
```



Answer:

```
0
```



Single Element Valid

```
target = 7  
nums = [7]
```



Answer:

```
1
```



Entire Array Needed

```
target = 15  
nums = [1,2,3,4,5]
```



Answer:

5



Time Complexity

Each element:

- enters window once
- leaves window once

Therefore:

```
Time:  $O(n)$   
Space:  $O(1)$ 
```



C++ Code

```
</> C++
```



```

class Solution {
public:
    int minSubArrayLen(int target, vector<int>& nums) {
        int n = nums.size();

        int left = 0;
        int sum = 0;
        int ans = INT_MAX;

        for(int right = 0; right < n; right++) {

            sum += nums[right];

            while(sum >= target) {

                ans = min(ans, right - left + 1);

                sum -= nums[left];
                left++;
            }

            return ans == INT_MAX ? 0 : ans;
        }
    };
}

```

5. Dry Run

Input

```

target = 7
nums = [2,3,1,2,4,3]

```



right = 0

```

sum = 2

```



Not valid.

right = 1

```

sum = 5

```



Not valid.

right = 2

```
sum = 6
```



Not valid.

right = 3

```
sum = 8
```



Valid.

```
window = [2,3,1,2]
```

```
length = 4
```

```
ans = 4
```



Shrink:

```
sum = 6
```

```
left = 1
```



right = 4

```
sum = 10
```



Valid.

```
length = 4
```

```
ans = 4
```



Shrink:

```
remove 3  
sum = 7  
length = 3  
  
ans = 3
```



Shrink again:

```
remove 1  
sum = 6  
left = 3
```



right = 5

```
sum = 9
```



Valid.

```
window = [2,4,3]  
length = 3
```



Shrink:

```
remove 2  
sum = 7  
  
length = 2  
ans = 2
```



Shrink:

```
remove 4  
sum = 3
```



Final Answer

```
2
```



Common Interview Trick

Many candidates miss the reason why Sliding Window works.

It works **only** because all elements are positive.

If negative numbers were allowed:

```
[2, -5, 10]
```



then:

```
Expanding window may decrease sum.  
Shrinking window may increase sum.
```



Sliding Window would no longer be valid.

That positivity constraint is the key observation interviewers look for.



Sources

Find K Closest Elements

Find K Closest Elements (LeetCode 658)

1. Problem Explanation (Simple English)

What is given?

- A sorted array `arr`.
- An integer `k`.
- An integer `x`.

What is required?

Return the `k` elements that are closest to `x`.

Rules:

1. Smaller distance from `x` is preferred.
2. If distances are equal, smaller value is preferred.
3. Final answer must be returned in **sorted order**.

Example

```
arr = [1,2,3,4,5]
k = 4
x = 3
```



Distances:

```
1 -> 2
2 -> 1
3 -> 0
4 -> 1
5 -> 2
```



Closest 4 elements:

```
[1,2,3,4]
```



Answer:

```
[1,2,3,4]
```



Why brute force may fail?

For every element:

- Compute distance from `x`
- Sort by distance

Sorting costs $O(n \log n)$.

We can do better because the array is already sorted.

2. Brute Force Approach

Idea

Store:

```
(distance from x, value)
```



for every element.

Sort by:

1. Smaller distance
2. Smaller value

Take first `k` elements.

Sort them again because answer must be in ascending order.

Time Complexity

Building pairs : $O(n)$



Sorting pairs : $O(n \log n)$

Sorting answer : $O(k \log k)$

Total : $O(n \log n)$

Space : $O(n)$

C++ Code

`</>` C++



```
class Solution {
public:
    vector<int> findClosestElements(vector<int>& arr, int k, int x) {

        vector<pair<int,int>> v;

        for(int num : arr) {
            v.push_back({abs(num - x), num});
        }

        sort(v.begin(), v.end());

        vector<int> ans;

        for(int i = 0; i < k; i++) {
            ans.push_back(v[i].second);
        }

        sort(ans.begin(), ans.end());

        return ans;
    }
};
```

3. Better Approach

Idea

Since array is sorted:

- Find position nearest to `x`.
- Use two pointers.
- Expand towards left/right choosing closer element.

Steps

1. Find insertion position using binary search.
2. Put:

```
left = pos - 1  
right = pos
```



3. Take k elements:
 - Compare distances.
 - Pick better side.
4. Sort result.

Time Complexity

```
Binary Search :  $O(\log n)$ 
```



```
Choose k elems:  $O(k)$ 
```

```
Sort answer :  $O(k \log k)$ 
```

```
Total :  $O(\log n + k \log k)$ 
```

```
Space :  $O(k)$ 
```

C++ Code

```
</> C++
```



```

class Solution {
public:
    vector<int> findClosestElements(vector<int>& arr, int k, int x) {

        int n = arr.size();

        int pos = lower_bound(arr.begin(), arr.end(), x) - arr.begin();

        int left = pos - 1;
        int right = pos;

        vector<int> ans;

        while(k--) {

            if(left < 0) {
                ans.push_back(arr[right++]);
            }
            else if(right >= n) {
                ans.push_back(arr[left--]);
            }
            else if(abs(arr[left] - x) <= abs(arr[right] - x)) {
                ans.push_back(arr[left--]);
            }
            else {
                ans.push_back(arr[right++]);
            }
        }

        sort(ans.begin(), ans.end());

        return ans;
    }
};

```

4. Optimal Approach (Binary Search on Window)

Core Observation / Trick

The final answer is always a **continuous window** of size k .

Example:

```

arr = [1,2,3,4,5]
k = 4

```



Possible windows:

```
[1,2,3,4]
```

```
[2,3,4,5]
```



We only need to find the best starting index.

Idea

Suppose window starts at `mid`.

Window:

```
arr[mid ... mid+k-1]
```



Compare:

```
x - arr[mid]
```



with

```
arr[mid+k] - x
```



Case 1

```
x - arr[mid] > arr[mid+k] - x
```



Right side is closer.

Move right.

```
left = mid + 1
```



Case 2

Otherwise:

```
right = mid
```



Steps

Binary search on window start.

Search range:

```
0 ... n-k
```



After finding start:

Return:

```
arr[start ... start+k-1]
```



Edge Cases

x smaller than all elements

```
arr=[1,2,3,4]
```

```
x=-5
```

```
answer=[1,2]
```



x larger than all elements

```
arr=[1,2,3,4]
```

```
x=100
```

```
answer=[3,4]
```



Exact Match

```
arr=[1,2,3,4,5]
```

```
x=3
```

```
answer centered around 3
```



Time Complexity

Binary search over:

0 ... n-k



Time : $O(\log(n-k) + k)$

Space : $O(1)$ (excluding output)



This is the best solution.

C++ Code

 C++



```
class Solution {
public:
    vector<int> findClosestElements(vector<int>& arr, int k, int x) {

        int n = arr.size();

        int left = 0;
        int right = n - k;

        while(left < right) {

            int mid = left + (right - left) / 2;

            if(x - arr[mid] > arr[mid + k] - x) {
                left = mid + 1;
            }
            else {
                right = mid;
            }
        }

        return vector<int>(arr.begin() + left,
                           arr.begin() + left + k);
    }
};
```

5. Dry Run

Input


```
arr = [1,2,3,4,5]
k = 4
x = 3
```



Search window start in:

```
0 ... 1
```



Iteration 1

```
left = 0
right = 1

mid = 0
```



Compare:

```
x - arr[mid]
= 3 - 1
= 2
```



and

```
arr[mid+k] - x
= arr[4] - 3
= 5 - 3
= 2
```



Check:

```
2 > 2 ?
No
```



So:

```
right = mid
= 0
```



Loop ends.

```
start = 0
```



Return:

```
arr[0..3]  
=  
[1,2,3,4]
```



Final Answer

```
[1,2,3,4]
```



Common Interview Trick

Most candidates use:

```
Sort by abs(arr[i] - x)
```



which gives $O(n \log n)$.

The hidden observation is:

```
Because arr is already sorted,  
the answer must form one contiguous window.
```



Once you realize that, the problem becomes a **Binary Search on Window Start Index**, giving:

```
 $O(\log(n-k) + k)$ 
```



which is the expected optimal solution.



Sources

Minimum Window Substring

Minimum Window Substring (LeetCode 76)

1. Problem Explanation (Simple English)

What is given?

- String `s`
- String `t`

What is required?

Find the **smallest substring** of `s` that contains **all characters** of `t` (including duplicates).

If no such substring exists, return:

`""`



Example

```
s = "ADOBECODEBANC"
t = "ABC"
```



Valid windows:

```
ADOBEC
DOBECODEBA
BANC
```



Smallest valid window:

```
BANC
```



Answer:

```
"BANC"
```



Important Note

Duplicates matter.

Example:

```
t = "AABC"
```



Then the window must contain:

```
A -> 2 times  
B -> 1 time  
C -> 1 time
```



Why brute force fails?

There are:

$O(n^2)$



substrings.

Checking every substring separately makes the solution very slow.

2. Brute Force Approach

Idea

Generate every substring.

For each substring:

- Count character frequencies.
- Check whether it contains all characters of `t`.
- Update minimum length.

Time Complexity

Number of substrings:

$O(n^2)$



Checking each substring:

$O(n)$



Total:

Time: $O(n^3)$
Space: $O(256)$



C++ Code

 C++



```

class Solution {
public:
    bool valid(vector<int>& freq, vector<int>& need) {
        for(int i = 0; i < 256; i++) {
            if(freq[i] < need[i]) return false;
        }
        return true;
    }

    string minWindow(string s, string t) {

        int n = s.size();

        vector<int> need(256, 0);

        for(char c : t)
            need[c]++;

        int bestLen = INT_MAX;
        int start = -1;

        for(int i = 0; i < n; i++) {

            vector<int> freq(256, 0);

            for(int j = i; j < n; j++) {

                freq[s[j]]++;

                if(valid(freq, need)) {

                    if(j - i + 1 < bestLen) {
                        bestLen = j - i + 1;
                        start = i;
                    }
                }
            }
        }

        return start == -1 ? "" : s.substr(start, bestLen);
    }
};

```

3. Better Approach

Idea

For every starting position:

- Extend window to the right.

- Maintain frequencies incrementally.
- As soon as window becomes valid:
 - Update answer.
 - Stop extending from this start.

This avoids rebuilding frequency arrays repeatedly.

Time Complexity

Time: $O(n^2)$
Space: $O(256)$



C++ Code

⌕ C++



```

class Solution {
public:
    string minWindow(string s, string t) {

        vector<int> need(256, 0);

        for(char c : t)
            need[c]++;

        int n = s.size();

        int bestLen = INT_MAX;
        int start = -1;

        for(int i = 0; i < n; i++) {

            vector<int> freq(256, 0);

            for(int j = i; j < n; j++) {

                freq[s[j]]++;

                bool ok = true;

                for(int k = 0; k < 256; k++) {
                    if(freq[k] < need[k]) {
                        ok = false;
                        break;
                    }
                }

                if(ok) {
                    if(j - i + 1 < bestLen) {
                        bestLen = j - i + 1;
                        start = i;
                    }
                    break;
                }
            }
        }

        return start == -1 ? "" : s.substr(start, bestLen);
    }
};

```

4. Optimal Approach (Sliding Window)

Core Observation / Trick

We need:

Smallest valid window



This is a classic:

Expand -> Make valid
Shrink -> Make minimum



pattern.

Idea

Maintain:

[left ... right]



Expand

Move `right` and include characters.

Valid Window

When all characters of `t` are present.

Shrink

Move `left` while window remains valid.

This guarantees the smallest window for that `right`.

Key Variable

Let:

`need[c]` = frequency required
`window[c]` = frequency inside current window



Maintain:

`formed`



Number of required characters currently satisfied.

When:

```
formed == t.length()
```



the window is valid.

Steps

1. Count frequencies of `t`.
 2. Expand `right`.
 3. If character contributes toward requirement:
 - increase `formed`.
 4. When window becomes valid:
 - update answer.
 - shrink from left.
 5. Continue.
-

Edge Cases

No Answer

```
s = "abc"  
t = "xyz"
```



Return:

```
""
```



Exact Match

```
s = "ABC"  
t = "ABC"
```



Return:

```
"ABC"
```



Duplicate Characters

```
t = "AABC"
```



Need 2 A's.

Frequency counts handle this automatically.

Time Complexity

Each character:

- enters window once
- leaves window once

Therefore:

```
Time: O(n)
```

```
Space: O(256)
```



C++ Code

```
</> C++
```



```

class Solution {
public:
    string minWindow(string s, string t) {

        vector<int> need(128, 0);

        for(char c : t)
            need[c]++;

        int left = 0;
        int formed = 0;

        int minLen = INT_MAX;
        int start = 0;

        for(int right = 0; right < s.size(); right++) {

            char c = s[right];

            need[c]--;

            if(need[c] >= 0)
                formed++;

            while(formed == t.size()) {

                if(right - left + 1 < minLen) {
                    minLen = right - left + 1;
                    start = left;
                }

                need[s[left]]++;

                if(need[s[left]] > 0)
                    formed--;

                left++;
            }
        }

        return minLen == INT_MAX
            ? ""
            : s.substr(start, minLen);
    }
};

```

5. Dry Run

Input

```
s = "ADOBECODEBANC"  
t = "ABC"
```



Need:

```
A = 1  
B = 1  
C = 1
```



Expand Right

```
ADOBEC
```



Now contains:

```
A, B, C
```



Valid window.

```
length = 6  
answer = "ADOBEC"
```



Shrink

Remove unnecessary characters.

Continue expanding.

Eventually:

```
BANC
```



Contains:

```
A = 1  
B = 1  
C = 1
```



Length:

4



Smaller than previous answer.

Update:

```
answer = "BANC"
```



No smaller valid window exists.

Final Answer

```
"BANC"
```



Common Interview Trick

Most candidates think:

```
formed = number of distinct characters matched
```



That is incorrect.

Example:

```
t = "AABC"
```



You need:

```
A = 2  
B = 1  
C = 1
```



The elegant trick used in the optimal solution is:

```
formed = total matched characters
```



and whenever:

```
formed == t.size()
```



the window is guaranteed to contain every required character with the correct frequency.

This leads to the clean $O(n)$ **Sliding Window** solution expected in interviews.

     ...  Sources

Sliding Window Maximum

Sliding Window Maximum (LeetCode 239)

1. Problem Explanation (Simple English)

What is given?

- An array `nums`.
- An integer `k`.

What is required?

For every window of size `k`, find the maximum element.

Return all maximums.

Example

```
nums = [1,3,-1,-3,5,3,6,7]
k = 3
```



Windows:

```
[1,3,-1]    -> 3
[3,-1,-3]   -> 3
[-1,-3,5]   -> 5
[-3,5,3]    -> 5
[5,3,6]     -> 6
[3,6,7]     -> 7
```



Answer:

```
[3,3,5,5,6,7]
```



Why brute force fails?

For every window:

- Scan all `k` elements.
- Find maximum.

If:

```
n = 100000  
k = 50000
```



Then $O(n \cdot k)$ is too slow.

2. Brute Force Approach

Idea

For every window:

- Traverse all `k` elements.
 - Compute maximum.
 - Store answer.
-

Time Complexity

```
Number of windows =  $O(n)$ 
```



```
Finding max in each window =  $O(k)$ 
```

```
Time =  $O(n \cdot k)$ 
```

```
Space =  $O(1)$ 
```

C++ Code

```
</> C++
```




```

class Solution {
public:
    vector<int> maxSlidingWindow(vector<int>& nums, int k) {

        int n = nums.size();

        vector<int> ans;

        for(int i = 0; i <= n - k; i++) {

            int mx = nums[i];

            for(int j = i; j < i + k; j++) {
                mx = max(mx, nums[j]);
            }

            ans.push_back(mx);
        }

        return ans;
    }
};

```

3. Better Approach

Idea

Use a Max Heap.

Store:

(value, index)



for elements inside the window.

The top of the heap always gives the maximum.

Remove elements that are outside the current window.

Steps

1. Insert current element into heap.
2. Remove outdated indices.
3. When window size reaches k:
 - Heap top is answer.

Time Complexity

Each insertion/removal:

$O(\log n)$



For all elements:

Time = $O(n \log n)$

Space = $O(n)$



C++ Code

 C++



```
class Solution {
public:
    vector<int> maxSlidingWindow(vector<int>& nums, int k) {

        priority_queue<pair<int,int>> pq;

        vector<int> ans;

        for(int i = 0; i < nums.size(); i++) {

            pq.push({nums[i], i});

            while(!pq.empty() && pq.top().second <= i - k) {
                pq.pop();
            }

            if(i >= k - 1) {
                ans.push_back(pq.top().first);
            }

        }

        return ans;
    }
};
```

4. Optimal Approach (Monotonic Deque)

Core Observation / Trick

We need:

Maximum of current window



Instead of storing all elements, store only useful candidates.

Use a deque that is:

Monotonically Decreasing



Meaning:

```
nums[dq[0]] >= nums[dq[1]] >= nums[dq[2]]
```



The front always contains the maximum element.

Idea

For every index **i**:

Remove Outdated Elements

If index is outside the window:

```
dq.front() <= i-k
```



remove it.

Remove Smaller Elements

Before inserting current element:

While:

```
nums[dq.back()] <= nums[i]
```



remove from back.

Why?

Because the new element is larger and will dominate them in future windows.

Insert Current Index

Push current index into deque.

Get Answer

When first complete window is formed:

```
i >= k-1
```



Answer:

```
nums[dq.front()]
```



Edge Cases

k = 1

```
nums = [1,2,3]
```



```
answer = [1,2,3]
```

All Equal

```
nums = [5,5,5]
```



```
answer = [5,5]
```

Strictly Increasing

```
1 2 3 4 5
```



Deque always keeps latest element.

Strictly Decreasing

5 4 3 2 1



Deque stores all indices in window.

Still $O(n)$.

Time Complexity

Why $O(n)$?

Each index:

- inserted once
- removed once

Total operations:

$O(n)$



Therefore:

Time = $O(n)$
Space = $O(k)$



C++ Code

⌞ C++



```

class Solution {
public:
    vector<int> maxSlidingWindow(vector<int>& nums, int k) {

        deque<int> dq;
        vector<int> ans;

        for(int i = 0; i < nums.size(); i++) {

            while(!dq.empty() && dq.front() <= i - k) {
                dq.pop_front();
            }

            while(!dq.empty() &&
                nums[dq.back()] <= nums[i]) {
                dq.pop_back();
            }

            dq.push_back(i);

            if(i >= k - 1) {
                ans.push_back(nums[dq.front()]);
            }
        }

        return ans;
    }
};

```

5. Dry Run

Input

```

nums = [1,3,-1,-3,5,3,6,7]
k = 3

```



i = 0

```

dq = [0]
value = [1]

```



i = 1

```
nums[1] = 3
```



Remove smaller:

```
1 < 3
```



```
dq = []
```



Push:

```
dq = [1]
```



Values:

```
[3]
```



i = 2

Push:

```
dq = [1,2]  
values = [3,-1]
```



Window complete.

Maximum:

```
nums[1] = 3
```



Answer:

```
[3]
```



i = 3

Push:

```
dq = [1,2,3]
```



Maximum:

3



Answer:

[3,3]



i = 4

Remove smaller elements:

-3 < 5
-1 < 5
3 < 5



Deque becomes empty.

Push 4:

dq = [4]



Maximum:

5



Answer:

[3,3,5]



Continue similarly.

Final answer:

[3,3,5,5,6,7]



Common Interview Trick

Most candidates know Heap solution:

$O(n \log n)$



But interviewers usually expect:

Monotonic Deque



The key observation is:

If `nums[i] >= nums[j]`
and `i` comes after `j`,



then `j` can never become
the maximum of any future window
that also contains `i`.

Therefore, smaller elements are removed immediately, leading to the optimal:

Time = $O(n)$
Space = $O(k)$



solution.



Sources